

PDS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 02 -

PADRÕES DE RESILIÊNCIA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 13

REFERÊNCIAS 14

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Você já conhece os fundamentos de uma arquitetura distribuída e quais arquiteturas padrões utilizar. Mas, como deixar essa arquitetura resiliente? Nesta aula, você descobrirá como implementar padrões de resiliência para proteger seu sistema de falhas, garantir alta disponibilidade e oferecer a melhor experiência para os usuários.

Vamos explorar os padrões Circuit Breaker, Bulkhead, Rate Limiting, entre outros, e como podem ser aplicados de maneira estratégica. Além disso, vamos analisar os prós e contras de cada um, ajudando você a entender qual padrão usar em diferentes cenários.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nas videoaulas a seguir, vamos aprofundar os conceitos sobre padrões de resiliência e analisar as estratégias mais adequadas para diferentes cenários. O foco estará em entender o funcionamento de cada padrão, comparando seus benefícios e limitações em situações práticas. Essas aulas vão preparar você para aplicar os padrões de forma mais consciente e estratégica.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Na aula anterior, nós exploramos o que é uma arquitetura distribuída. Esse conceito organiza sistemas em componentes independentes que se comunicam por meio de redes, permitindo que equipes escalem e mantenham partes específicas do sistema sem impactar o todo. Essa abordagem é amplamente utilizada por empresas como Netflix e Amazon, que implementaram arquiteturas distribuídas para lidar com milhões de usuários simultaneamente.

Por outro lado, vimos que essa estrutura apresenta desafios: a latência, ocasionada por múltiplos saltos na rede; a dificuldade em manter a consistência dos dados entre serviços; e a vulnerabilidade a falhas em um único componente que podem causar efeitos em cascata. Esses problemas tornam claro que um sistema resiliente é essencial para garantir a continuidade das operações.

O autor Michael T. Nygard, em "Release It!: Design and Deploy Production-Ready Software", apresenta padrões como **Circuit Breaker** e **Bulkhead** para lidar com falhas de forma inteligente, prevenindo que pequenos problemas se transformem em grandes crises.

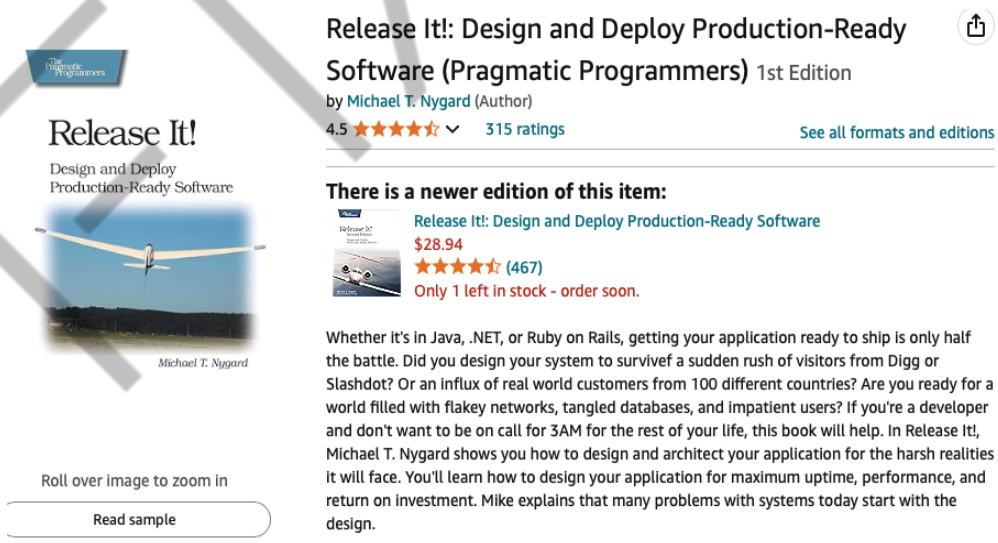


Figura 1 – Livro Release it! Design and Deploy Production-Ready
Fonte: AWS (2017)

Complementando, Martin Kleppmann, em "Designing Data-Intensive Applications", explora como arquiteturas resilientes podem lidar com grandes volumes de dados em sistemas distribuídos.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O software não existe apenas para ser um produto final; ele é uma ferramenta para apoiar seus processos de negócios e garantir a satisfação dos clientes. No entanto, um software que não está rodando em produção não gera valor algum. Mesmo assim, um software produtivo precisa ser mais do que funcional: ele deve ser correto, confiável e sempre disponível.

Quando falamos em resiliência no desenvolvimento de software, o objetivo central é criar componentes capazes de suportar falhas, tanto dentro do seu próprio escopo quanto de outros componentes dos quais dependem. Técnicas como failover automático e redundância ajudam a mitigar problemas, mas, na prática, quase todos os sistemas modernos são distribuídos. Até mesmo uma aplicação web básica pode incluir servidores web, bancos de dados, firewalls, proxies, balanceadores de carga e servidores de cache. Isso sem mencionar a infraestrutura de rede, que é composta por inúmeros elementos onde falhas inevitavelmente ocorrem.

Além de falhas completas, serviços também podem sofrer com respostas lentas ou até mesmo respostas incorretas, mas que tecnicamente seguem o formato esperado. Quanto mais complexo o sistema, maior a chance de algo dar errado em algum ponto.

A disponibilidade, por sua vez, é um dos atributos de qualidade mais valorizados. Ela mede quanto tempo um componente está realmente disponível em relação ao tempo que ele deveria estar acessível. Esse indicador pode ser representado por uma fórmula que reflete sua confiabilidade operacional.

No mundo da tecnologia, a resiliência é a chave para a sobrevivência e o sucesso. Vamos explorar alguns padrões de design de arquitetura de software que ajudam a criar sistemas mais robustos e tolerantes a falhas. Nesta aula, apresentaremos os principais padrões de resiliência, como Circuit Breaker, Bulkhead, Rate Limiting, Retry e Timeout, discutindo suas aplicações práticas, vantagens, limitações e como integrá-los para criar sistemas mais robustos.

Circuit Breaker

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

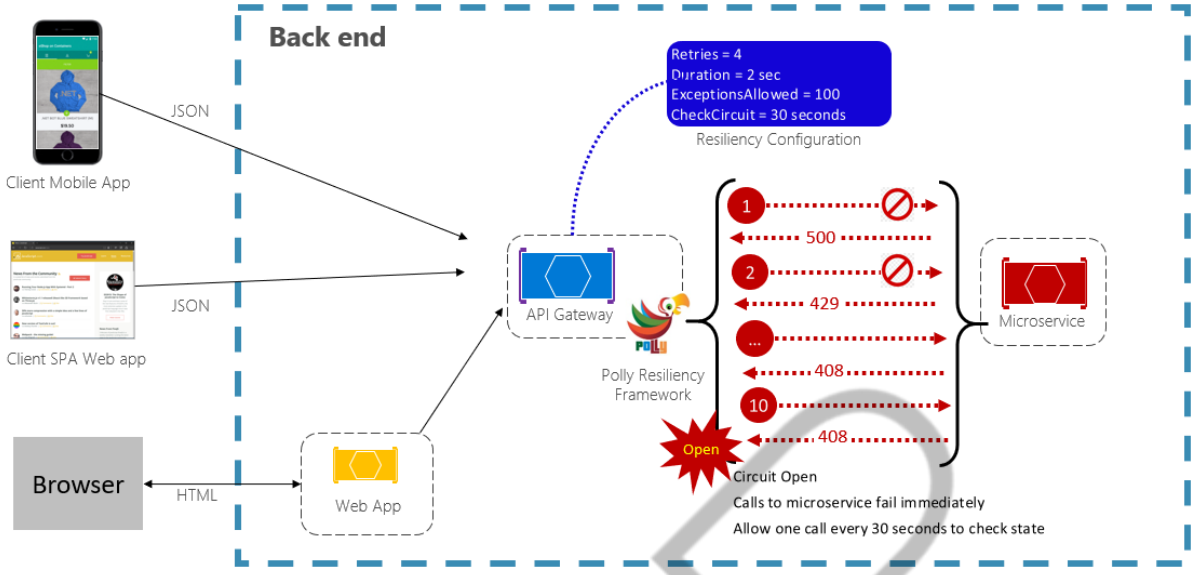


Figura 2 – Padrão de disjuntor
Fonte: Microsoft Learn (2023)

O padrão Circuit Breaker é inspirado no disjuntor elétrico, interrompendo fluxos que apresentam falhas repetidas. Em um sistema distribuído, ele monitora as chamadas entre serviços e interrompe requisições quando detecta uma alta taxa de erros.

O funcionamento é baseado em três estados: fechado, onde as requisições fluem normalmente; aberto, que bloqueia novas tentativas por um tempo; e meio aberto, onde algumas requisições são permitidas para testar a recuperação do serviço. Essa abordagem previne que serviços falhos impactem sistemas dependentes.

Além de evitar sobrecarga, o Circuit Breaker melhora a experiência do usuário ao evitar tempos de espera longos e falhas recorrentes. Implementações como Resilience4j ou Hystrix tornam a aplicação prática deste padrão relativamente simples.

Exemplo: imagine um sistema que consulta uma API externa. Se a API começar a falhar, o Circuit Breaker entra no estado "aberto" e interrompe as chamadas, evitando que o sistema continue enviando requisições que provavelmente falharão.

Prós: previne falhas em cascata, reduz a latência ao evitar tentativas inúteis e melhora a estabilidade geral do sistema.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Contras: pode ser complexo configurar os limites corretos e requer monitoramento contínuo para ajustes.

Bulkhead

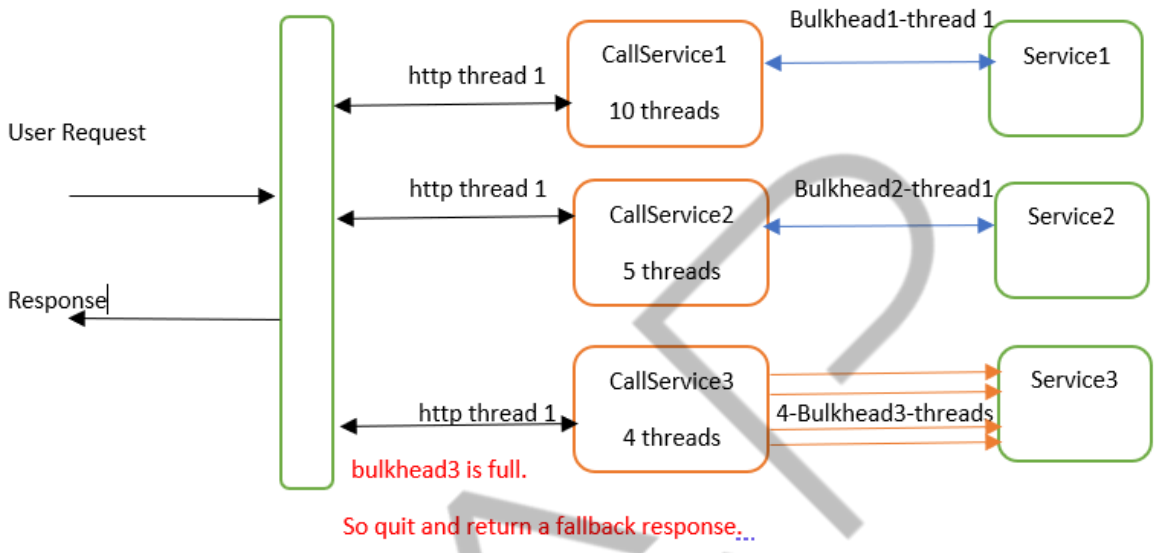


Figura 3 – Bulkheads
Fonte: jession4d.com (2023)

O padrão Bulkhead isola diferentes partes do sistema para que uma falha não cause impacto em toda a operação. Ele aloca recursos, como threads ou conexões, para grupos específicos de operações.

Esse isolamento protege o sistema ao evitar que falhas em um componente consumam todos os recursos disponíveis. A aplicação do Bulkhead é particularmente útil em sistemas que atendem múltiplos clientes ou serviços. Ele combina bem com outros padrões, como Circuit Breaker, para fornecer resiliência adicional.

Exemplo: em um sistema de e-commerce, você pode alocar threads específicas para o processamento de pagamentos e outras para o cadastro de novos usuários. Assim, uma sobrecarga no cadastro não afetará as compras.

Prós: aumenta a confiabilidade ao limitar o impacto de falhas. Facilita a priorização de operações críticas.

Contras: requer planejamento detalhado para alocação de recursos e pode desperdiçar recursos subutilizados em compartimentos específicos.

Rate Limiting

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

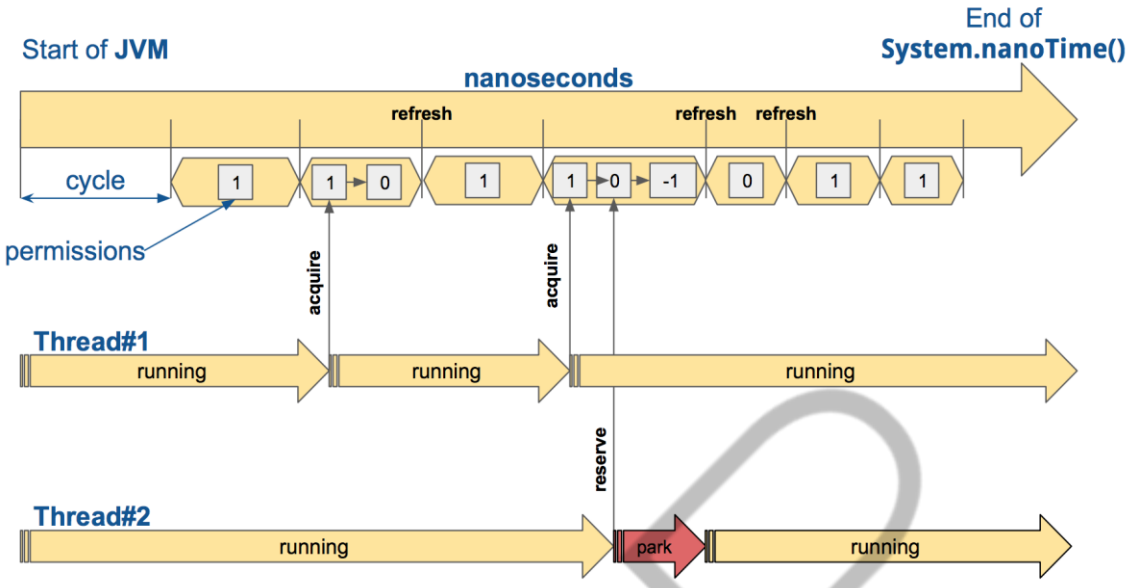


Figura 4 – Rate limit
Fonte: resilience4j.com (2023)

O Rate Limiting protege os serviços de sobrecarga controlando a quantidade de requisições que podem ser processadas em um período de tempo. É amplamente usado em APIs públicas para evitar abusos de recursos.

Esse padrão funciona através de estratégias como o Token Bucket, onde cada requisição consome um token disponível. Quando os tokens acabam, novas requisições são negadas ou enfileiradas até que mais tokens sejam gerados. Ferramentas como NGINX ou Redis são comumente usadas para implementar Rate Limiting. Além de proteger o sistema, ele pode ser usado para oferecer experiências justas a diferentes usuários, garantindo que ninguém monopolize os recursos.

Exemplo: uma API pública permite apenas 100 requisições por minuto por cliente. Requisições excedentes recebem uma resposta de erro, protegendo o sistema contra abuso.

Prós: protege contra ataques de negação de serviço (DDoS) e mantém os serviços operando sob alta demanda.

Contras: pode frustrar usuários legítimos durante picos de uso e requer políticas claras para lidar com limites excedidos.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Timeout

O padrão Timeout define limites de tempo para operações, garantindo que o sistema não fique aguardando indefinidamente por uma resposta. Isso é essencial para liberar recursos de forma eficiente e evitar bloqueios em cascata.

Por exemplo, ao consultar um serviço externo, um timeout de 3 segundos pode ser configurado. Se o serviço não responder nesse tempo, a chamada será encerrada e tratada como falha. Frameworks modernos permitem configurar timeouts em diferentes níveis, desde conexões de rede até chamadas de API. Além de prevenir sobrecarga, o Timeout melhora a experiência do usuário ao evitar esperas longas e desnecessárias.

Exemplo: um serviço de busca em um site de notícias cancela operações que ultrapassem 2 segundos, devolvendo um resultado parcial ou uma mensagem de erro ao usuário.

Prós: libera recursos rapidamente e evita gargalos causados por serviços lentos.

Contras: requer ajustes precisos para evitar timeouts excessivamente curtos ou longos.

Failover

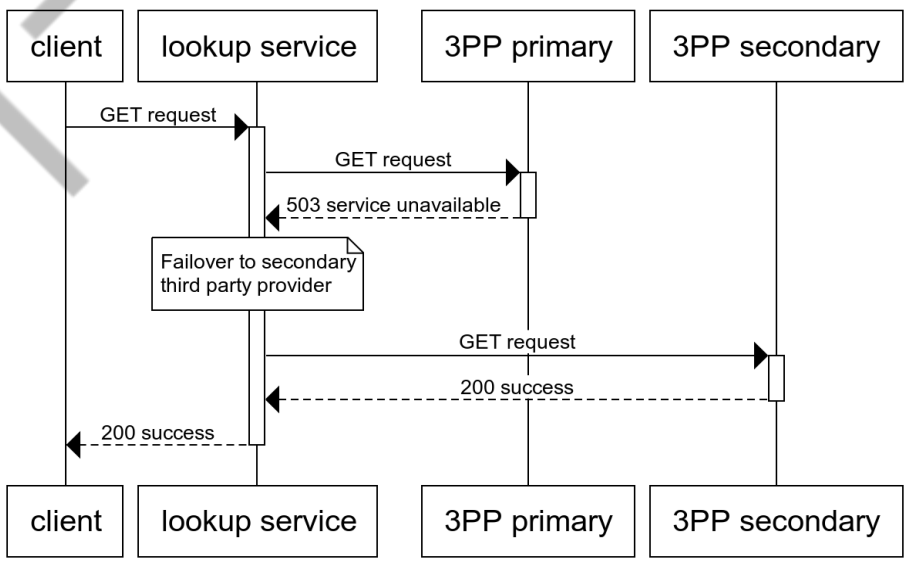


Figura 5 – Failover
Fonte: resilience4j.com (2023)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O Failover é um padrão que garante alta disponibilidade ao redirecionar automaticamente o tráfego para instâncias ou serviços redundantes em caso de falha. Ele é comum em balanceadores de carga e clusters de banco de dados.

Em sistemas distribuídos, o Failover pode ser configurado para detectar falhas em tempo real e ativar backups ou réplicas. Isso reduz o tempo de inatividade e melhora a confiabilidade do sistema. Serviços como AWS e Google Cloud oferecem soluções de Failover integradas. Esse padrão é especialmente importante em sistemas críticos, onde a indisponibilidade pode ter impactos significativos para os negócios.

Exemplo: um cluster de banco de dados detecta falha em um nó primário e transfere as operações para um nó secundário.

Prós: garante alta disponibilidade e continuidade do serviço.

Contras: configurações inadequadas podem causar problemas como "failover em loop" ou inconsistência de dados.

Throttling

O Throttling ajusta dinamicamente a taxa de requisições permitidas com base na carga do sistema. Ao contrário do Rate Limiting, ele é mais adaptável e protege o sistema contra picos inesperados.

Quando o sistema detecta sobrecarga, o Throttling reduz as requisições processadas ou retorna respostas como "429 Too Many Requests". Isso garante que operações essenciais continuem funcionando enquanto as não essenciais são limitadas.

Esse padrão é eficaz em serviços altamente dinâmicos, onde a carga pode variar rapidamente, e é frequentemente usado junto com Rate Limiting.

Exemplo: um serviço de streaming ajusta automaticamente a qualidade do vídeo ou reduz o número de conexões simultâneas durante picos de demanda.

Prós: protege o sistema contra sobrecarga e garante que operações críticas continuem funcionando.

Contras: pode degradar a experiência do usuário para operações não críticas.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Sistemas distribuídos frequentemente requerem a combinação de padrões de resiliência para alcançar maior robustez. Por exemplo, um sistema pode usar **Rate Limiting** para evitar sobrecarga inicial, enquanto o **Circuit Breaker** protege serviços internos contra falhas repetidas e o **Bulkhead** isola recursos críticos.

Concluímos que a combinação de padrões de resiliência, adaptados às características específicas do sistema e aos seus pontos de vulnerabilidade, é essencial para alcançar uma operação confiável e eficiente.

O monitoramento contínuo e ajustes finos permitem que os padrões trabalhem de forma integrada, melhorando a experiência do usuário, reduzindo o impacto de falhas e otimizando custos operacionais. Em um cenário de sistemas cada vez mais complexos, a resiliência se torna um pilar fundamental para a competitividade das organizações.

Na próxima aula, vamos explorar como implementar resiliência com sistemas Java utilizando ferramentas e bibliotecas especializadas, como o **Resilience4j**. Vamos entender na prática como aplicar estratégias como Circuit Breaker, Retry e Bulkhead para construir sistemas resilientes.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a construir uma arquitetura de sistema distribuído com as melhores práticas de resiliência. Exploramos como esses padrões são essenciais para garantir a estabilidade e a continuidade de sistemas diante de falhas e alta demanda.

Você conheceu diferentes tipos de padrões de resiliência, como **Circuit Breaker**, **Bulkhead**, **Rate Limiting**, entre outros, e descobriu como aplicá-los de maneira estratégica. Cada padrão foi apresentado com exemplos que conectam teoria e realidade, além de uma análise clara de seus prós e contras, destacando a importância de escolher a abordagem certa para cada contexto.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

AMAZON WEB SERVICES. **Circuit Breaker. AWS Prescriptive Guidance.** Disponível em: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html. Acesso em: 12 dez. 2024.

D'SOUZA, **R. A quick guide to Resilience4j with Spring Boot. JSession4D**. Disponível em: https://jsession4d.com/a-quick-guide-to-resilience4j-with-spring-boot/. Acesso em: 12 dez. 2024.

KLEPPMANN, M. **Designing Data-Intensive Applications.** Sebastopol: O'Reilly Media, 2017.

LYDTECH CONSULTING. **Blog: Resilience4j. Lydtech Consulting**. Disponível em: https://www.lydtechconsulting.com/blog-resilience4j.html. Acesso em: 12 dez. 2024.

NATTER, M. **Application resiliency patterns. Microsoft Learn**. Disponível em: https://learn.microsoft.com/pt-br/dotnet/architecture/cloud-native/application-resiliency-patterns. Acesso em: 12 dez. 2024.

NORMAN, M. **Release It!: Design and Deploy Production-Ready Software.** Dallas: Pragmatic Bookshelf, 2007.

RESILIENCE4J. **Getting Started. Resilience4j Documentation**. Disponível em: https://resilience4j.readme.io/docs/getting-started. Acesso em: 12 dez. 2024.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Resiliência. Confiança. Escalabilidade.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com